Lesson 9 — Serial Communication (USART)

ATmega328P / Arduino Uno | Source: lecture 09 slides (Systems 16–17), Exercise 7 | Builds on Lessons 3 (ports), 4 (polling, interrupts) and 5 (ADC)

WHAT YOU WILL BE ABLE TO DO AFTER THIS LESSON

- Explain why embedded systems use **serial** (not parallel) and **asynchronous** communication.
- Draw an asynchronous **frame** and explain why both sides must agree on the **baud rate**.
- Explain the **line driver** (RS-232 ↔ TTL) and the **UART-to-USB** chips on Arduino boards.
- Explain how the USART **transmitter** and **receiver** work (UDR, shift registers, UDRE / TXC / RXC flags).
- Calculate **UBRR** and the **baud rate error**, and explain the **UL** suffix.
- Write the echo ("char encloser") in C with polling and with interrupts, in Arduino, and the Exercise 7 temp logger.

GIVEN TO YOU IN THE EXAM

The register sheet shows the bit positions of `UCSR0A/B/C`, `UBRR0` and `UDR0`; the pin sheet shows **D0 = PD0 = RXD** and **D1 = PD1 = TXD**. Check your printed copies. **The formulas, the frame, the flag meanings and the "why" answers below are "learn it".**

1. Why serial? Why not parallel?

Embedded systems talk to **other systems**: a PC (data logging, debugging), or another embedded system (an RC car or aircraft talks to its remote controller over radio: two embedded systems, wireless, but still serial rules).

Serial sends **one bit at a time**, one after another, so it is slower. **Parallel** sends several bits at once on several wires, so it is faster. So why serial?

THE MAIN REASON: I/O PINS ARE PRECIOUS

8-bit parallel needs 8 wires to send + 8 to receive = **16 I/O pins**. Serial needs only **2** (TX and RX). The ATmega328P has only 23 I/O pins; 16 for communication would leave 7 for all the sensors and actuators. A tiny 8-pin ATtiny would have nothing left. Pins are needed for sensors and actuators, so embedded systems use serial.

Second problem of parallel: crosstalk. Current in a wire makes a magnetic field that induces a small current in the wire next to it. A 0 sent on one wire can arrive looking like a 1. It gets worse with long wires. Twisting wire pairs (twisted-pair / UTP network cable) reduces it, but the pin problem remains.

2. Synchronous vs asynchronous

	Synchronous	Asynchronous
Wires	data **+ a clock wire** shared by both sides	data only (TX, RX), **no clock**
How the receiver knows when a bit arrives	one bit per clock pulse	both sides **agree on the data rate (baud rate)** in advance; start/stop bits mark each character
Receiver must know the sender's speed?	no	**yes**
Used	sometimes (SPI, I²C)	**most common in embedded systems** (UART), saves a pin

Why the receiver must line up with the sender: it receives a stream of bits and has to know where each character **starts and ends** (LSB to MSB) to turn 8 bits back into one ASCII character. It can do that either from a shared clock (synchronous) or from a known rate plus start/stop bits (asynchronous).

3. The asynchronous frame

Each character is **framed** between a **start bit (always 0)** and a **stop bit (always 1)**, then sent **LSB first**. The idle line sits at 1. 8 data bits + start + stop = a **10-bit frame**. Start and stop are called **synchronization (sync) bits**.

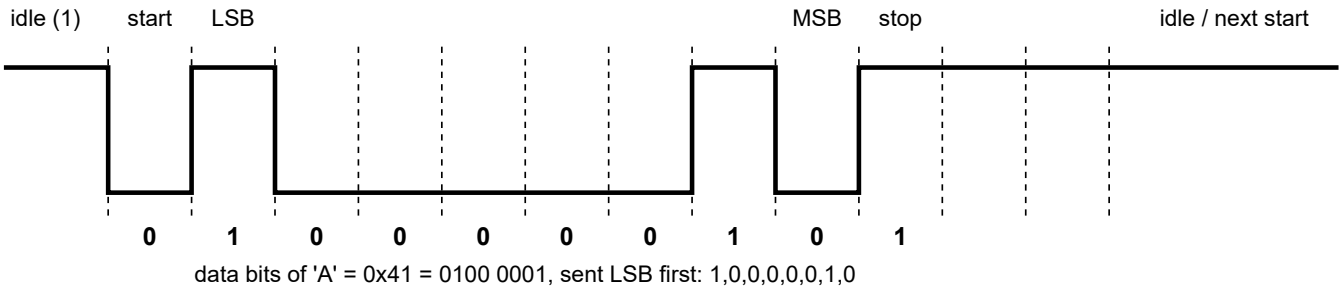


Figure 1. One 10-bit frame carrying 'A'. Time runs left to right; each box lasts 1/baud seconds.

- **Baud rate** = bits per second (bps), named after the French engineer Émile Baudot. 9600 bps = 960 frames = **960 characters per second**.
- Character size can be **5 to 9 bits**: 8 is normal; 9 = 8 data + 1 **parity** bit for error checking (11-bit frame); 5 = a hex digit (4 bits) + parity (7-bit frame).
- **How the receiver decodes** (lecturer's 1 bps example): the line drops to 0 → "start bit"; it starts counting; bits 1–8 are data; at the **10th** bit time it expects a **1** (stop). If it sees 1, it accepts the 8 bits as a character.
- **If the speeds don't match** (e.g. sender 10% slow): at the 10th bit time the receiver reads the MSB instead of the stop bit. If the MSB is 1 it accepts a **wrong character**; if 0 it detects a framing error and the output is garbage. This is why a wrong baud rate in the Serial Monitor prints **weird characters**.

4. The USART and why it is a separate peripheral

USART = Universal Synchronous/Asynchronous Receiver Transmitter (UART when used asynchronously). It is a peripheral on the chip, like the timers and ADC, with a **transmitter (TX)** and a **receiver (RX)**. **Why separate hardware?** The CPU is a single, slow (MHz), low-power core. If the USART does the bit-by-bit sending and receiving, the CPU is free for computation. It is the same reason we use hardware timers.

The ATmega328P has **one** USART, so its registers end in **0** (`UBRR0`, `UDR0`…). Chips with several (the ATmega2560 on the Arduino Mega has four) use 0, 1, 2, 3; writing the number into the name makes code easy to port.

Pins: RXD = **PD0 = D0**, TXD = **PD1 = D1**. Setting RXEN0/TXEN0 overrides their GPIO function (the "pin control" block). So **connect nothing else to D0/D1** while using serial; if serial misbehaves, check those two pins first. If you don't need serial, they are normal I/O pins.

5. Connecting to a PC

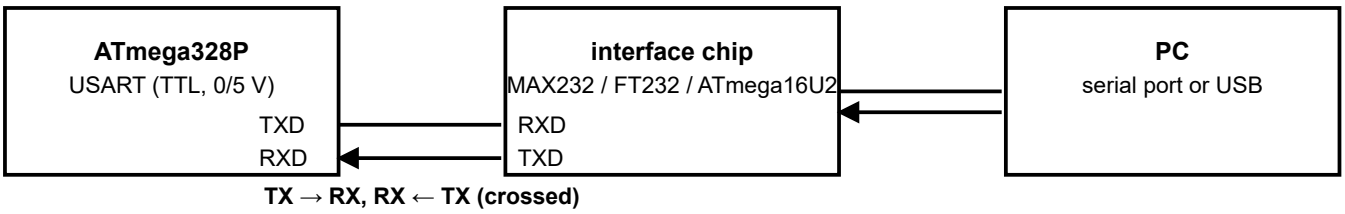


Figure 2. The transmitter of one side always goes to the receiver of the other, like the microphone of one phone feeding the speaker of the other.

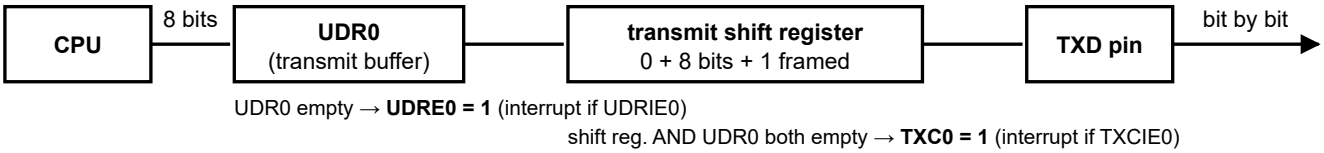
Option	Chip	Board	Notes
Old **serial port** (DB9, RS-232)	**MAX232 line driver**	Arduino Severino	Needs a separate 5 V supply.
USB, dedicated chip	**FT232RL** (FTDI)	Duemilanove (2009), old Uno	Needs the FTDI **driver** installed on each PC (ftdichip.com). TX/RX LEDs; powered from USB.
USB, a second microcontroller	**ATmega16U2** acting as USB↔UART	**Uno R3** (your board)	No extra driver. It has its own **crystal** because USB needs an accurate clock; the main chip uses a cheaper ceramic resonator.

WHY A LINE DRIVER? VOLTAGE LEVELS

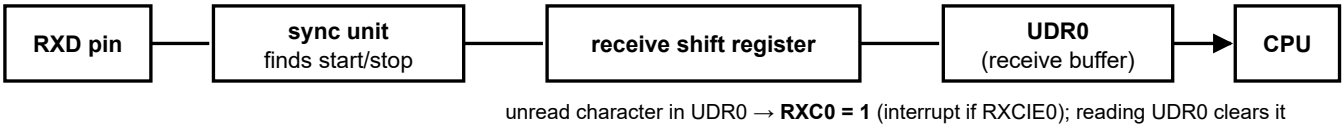
RS-232 comes from the 1960s mainframe era: logic **1 = −3 to −25 V**, logic **0 = +3 to +25 V**. The microcontroller uses **TTL**: 1 = 2–5 V, 0 = 0–0.8 V. Connecting them directly would mismatch and damage the chip, so a line driver (MAX232) converts TTL ↔ RS-232 both ways. The USB chips do the voltage *and* the protocol conversion.

6. How the transmitter and receiver work

TRANSMITTER (parallel in, serial out)



RECEIVER (serial in, parallel out)



Both sides: a baud rate generator, baud = F_CPU / (16 × (UBRR0 + 1)).
Transmit and receive buffers share one address and one name, UDR0: **write = transmit buffer, read = receive buffer**.

Figure 3. USART transmitter and receiver. The shift registers turn 8 parallel bits into a serial stream and back.

- **Transmit:** the CPU writes a character to UDR0. When the shift register is free, the character moves into it (so UDR0 empties and **UDRE0** is set), gets framed, and is shifted out on TXD at the baud rate. Writing a new character clears UDRE0. **UDRE0 is 1 after reset**, because the buffer starts empty.
- **Receive:** the sync unit recognises the start/stop bits, the bits fill the receive shift register, the character (without start/stop) moves to UDR0, and **RXC0** is set. Reading UDR0 clears it.
- Writing to UDR0 while it is **not empty**: the new character is **ignored** (lost). Reading UDR0 without waiting for RXC0 returns the **same old character** again.

UDRE OR TXC: WHICH GIVES MAXIMUM THROUGHPUT? [EXAM]

UDRE. It is set as soon as the *buffer* is free, so the next character waits in UDR0 while the previous one is still shifting out. TXC waits until *both* the buffer and the shift register are empty, which leaves a gap.

THEN WHAT IS TXC FOR? [EXAM]

Knowing that **everything really gone out**: (1) **half-duplex** links, where you release the line only when finished, like saying "over" on a walkie-talkie; (2) **power saving**: switch off the USART only after the last bit has left, e.g. a battery-powered sensor behind enemy lines.

7. The USART registers bit by bit

`UCSR0A`: status flags

	7	6	5	4	3	2	1	0	
UCSR0A	RXC	TXC	UDRE	·	·	·	·	·	receive complete, transmit complete, data register empty

`UCSR0B`: interrupt enables, RX/TX enable, size bit 2

UCSR0B	RXCIE	TXCIE	UDRIE	RXEN	TXEN	UCSZ2	·	·	all 0 after reset (RX and TX off to save power)

`UCSR0C`: character size bits 1–0

UCSR0C	·	·	·	·	·	UCSZ1	UCSZ0	·	8-bit = UCSZ2:0 = 011 (the default)

UCSZ2 UCSZ1 UCSZ0	000 = 5-bit	001 = 6	010 = 7	**011 = 8**	111 = 9

`UBRR0` is 16 bits wide but only the lower **12** bits are used (0–4095). **Why enable/disable RX and TX?** (1) to **save power**; (2) to choose between serial and **GPIO** use of D0/D1.

8. Baud rate: UBRR, the UL suffix, and the baud rate error

FORMULAS (NORMAL SPEED)
baud = F_CPU / (16 × (UBRR0 + 1)) ⇒ **UBRR0 = F_CPU / (16 × baud) − 1**
error % = (actual baud − desired baud) / desired baud × 100 (keep below about **2.5%**)

Step (9600 bps at 16 MHz)	Result
UBRR0 = 16 000 000 / (16 × 9600) − 1	103.17 → register holds integers only → **103**
Actual baud = 16 000 000 / (16 × 104)	**9615.38 bps**
Error = (9615.38 − 9600) / 9600	**0.16%** (OK)
With an **18.432 MHz** crystal: 18 432 000 / (16 × 9600) − 1	exactly **119** → **0% error**

Standard rates: 2400, 4800, **9600** (most common, the Arduino default), 14400, 19200, 28800, 38400, 57600, 76800, 115200, 230400, 460800, 921600 bps. Both ends must use the same one.

WHY THE UL SUFFIX IN `UBRR0 = 16000000 / (16 * 9600UL) - 1;` [EXAM]
In avr-gcc, plain numbers are `int` = **16 bits**. 16 × 9600 = 153 600 needs 20 bits, so as int only the lower 16 bits survive: **22 528**. Then 16 000 000 / 22 528 − 1 = **709**, giving a completely wrong baud rate. Writing `9600UL` makes one operand **unsigned long (32-bit)**, so the multiplication is done in 32 bits and gives 153 600, and UBRR0 = 103. It is a **logic error, not a compile error**: the compiler won't warn you.

Why errors matter: the receiver samples at its own rate. With a 10% error at 1 bps, the sender falls 1 s behind every 10 s, so the receiver reads the MSB where it expects the stop bit, giving a wrong character or garbage. Above about 2.5% the link is unreliable. For **critical** systems (missile guidance, a drone over a crowd) the target is **0%**: use a "baud-rate" crystal such as 18.432 MHz. Changing F_CPU changes every timer, PWM and ADC calculation too.

9. System 16: char encloser (polling)

The PC sends a character, the board replies with it in brackets: send `a`, get `[a]`. No wiring needed: just the Uno and its USB cable.

```
// charEncloserC.c
#include <avr/io.h>

void transmitChar(char ch)
{
    while (!(UCSR0A & 0b00100000));      // wait until UDRE0 = 1 (buffer empty)
    UDR0 = ch;                           // write character to transmit
}

int main(void)
{
    UBRR0 = 16000000 / (16 * 9600UL) - 1; // 103 -> 9600 bps
    UCSR0B = UCSR0B & 0b11111011;         // UCSZ02 = 0
    UCSR0C = UCSR0C | 0b00000110;         // UCSZ01 = 1, UCSZ00 = 1 -> 8-bit characters
    UCSR0B = UCSR0B | 0b00011000;         // RXEN0 = 1, TXEN0 = 1
    while (1)
    {
        while (!(UCSR0A & 0b10000000));  // wait until RXC0 = 1 (new character)
        char ch = UDR0;                  // read it (clears RXC0)
        transmitChar('[');
        transmitChar(ch);
        transmitChar(']');
    }
    return 0;
}
```

How `while (!(UCSR0A & 0b10000000));` **works:** the AND keeps only bit 7 (RXC0). While RXC0 = 0 the result is 0, `!0` = 1, and the empty loop repeats; when RXC0 = 1 the result is non-zero, `!` makes it 0, and the loop ends.
Lecturer's test questions: remove the UDRE wait in `transmitChar` and send `a`: you get only `[a`. '[' goes straight to the shift register, 'a' waits in UDR0, and ']' is written while UDR0 is still full, so it is ignored. Remove the RXC wait in `main`: the loop reads UDR0 over and over and keeps sending the old character, so try it and see.

ARDUINO VERSION

```
void setup() { Serial.begin(9600); }
void loop() {
 if (Serial.available()) {
  char ch = Serial.read();
  Serial.print('['); Serial.print(ch);
Serial.print(']');
 }
}
```

`begin` sets UBRR0 = 103, 8-bit, RX/TX on.
`available()` returns the number of unread characters.

MORE SERIAL FUNCTIONS

`println` adds `\r\n` (cursor to the next line); `readString()` reads a whole string; `print(78, BIN)` → 1001110, `print(78, HEX)` → 4E, `print(1.2345, 3)` → 1.234 (3 decimal places, cut off); `Serial.end()` turns RX/TX off so D0/D1 become normal I/O again.

Terminal emulator: software that imitates a 1960s "dumb" text terminal (TTY = teletype): keyboard + screen, 24 lines of 80 characters. Used for **debugging, sensor calibration, module configuration, data logging**. Examples: the Arduino **Serial Monitor**, or "Terminal for Atmel Studio" (Tools → Extensions and Updates), which can also save to a file. Choose the **COM port and the baud rate** to match. **CR** (`\r`, carriage return: back to the start of the line) and **LF** (`\n`, line feed / new line) come from typewriters; the terminal can append them to what you send ("No line ending" sends neither).

10. System 17: char encloser with interrupts

```
// charEncloserUsingInterruptsC.c
#include <avr/io.h>
#include <avr/interrupt.h>

char ch[] = {'[', ' ', ']'};            // middle element will hold the received character
volatile int i = 0;                     // array index, shared with the ISRs

int main(void)
{
    UBRR0 = 103;                        // 9600 bps
    UCSR0B = UCSR0B & 0b11111011;       // 8-bit characters
    UCSR0C = UCSR0C | 0b00000110;
    UCSR0B = UCSR0B | 0b00011000;       // enable RX and TX
    UCSR0B = UCSR0B | 0b10000000;       // RXCIE0 = 1: receive complete interrupt
    sei();                              // global interrupts
    while (1) { }                       // CPU free for other work
    return 0;
}

ISR(USART_RX_vect)                      // a character has arrived
{
    ch[1] = UDR0;                       // store it in the middle
    UCSR0B = UCSR0B & 0b01111111;       // stop receiving interrupts until it is sent
    UCSR0B = UCSR0B | 0b00100000;       // UDRIE0 = 1: start transmitting
}

ISR(USART_UDRE_vect)                    // UDR0 is empty: send the next character
{
    UDR0 = ch[i++];                     // send ch[i], then i = i + 1
    if (i == 3)                         // '[', char and ']' all written
    {
        i = 0;
        UCSR0B = UCSR0B & 0b11011111;   // UDRIE0 = 0
        UCSR0B = UCSR0B | 0b10000000;   // RXCIE0 = 1 again
    }
}
```

The receive interrupt is switched off while the bracketed character is being sent, so a second incoming character waits in the hardware buffer instead of overwriting `ch[1]`. `i` is `volatile` because the ISRs change it.

RECEIVE chain	TRANSMIT chain
bits arrive on RXD → receive shift register assembles them → character moved to UDR0 → RXC0 = 1 → if RXCIE0, `USART_RX_vect` runs → CPU reads UDR0 → RXC0 = 0	UDR0 empty → UDRE0 = 1 → if UDRIE0, `USART_UDRE_vect` runs → CPU writes UDR0 → UDRE0 = 0 → character moves to the shift register → UDRE0 = 1 → bits leave on TXD

11. Build example: Exercise 7 — the temp logger

Task: LM35 on **A0**, Uno on USB. When the board receives **'Y'** it sends the temperature **every second**; any other character stops it. A C# (.NET) program on the PC has **Start** (sends 'Y') and **Stop** (sends 'N') buttons and shows the temperature.

Hardware: LM35 +Vs → 5V, Vout → A0, GND → GND (as in Lesson 5). Nothing on D0/D1. USB to the PC.

Q1: Arduino version

```
char ch = 'N';

void setup() { Serial.begin(9600); }

void loop()
{
    if (Serial.available())
        ch = Serial.read();          // latest command
    if (ch == 'Y')
    {
        int temp = analogRead(A0) * 5.0 / 1024 * 100;
        Serial.println(temp);        // e.g. "30\r\n"
        delay(1000);
    }
}
```

Q3: why `char str[4]`?

The LM35 range is −55 to 150 °C. The longest strings `itoa` can make are **"−55"** and **"150"**: 3 characters, plus the **'\0'** terminator that ends every C string = **4**. With 3 elements the terminator would overwrite the next variable in memory.

Note

`delay(1000)` means a Stop may take up to 1 s to act. For a truly immediate stop, time with `millis()` (or a timer, Lesson 7) instead of `delay`.

Q2: the blanks of the C version (labeled constants):

```
// initADC()
ADMUX  = ADMUX | (1<<REFS0);                                    // Vref = AVcc (5 V)
ADMUX  = ADMUX & ~((1<<MUX3)|(1<<MUX2)|(1<<MUX1)|(1<<MUX0)); // channel ADC0 (PC0)
ADCSRA = ADCSRA | (1<<ADPS2)|(1<<ADPS1)|(1<<ADPS0);         // /128 -> 125 kHz
ADCSRA = ADCSRA | (1<<ADEN);                                    // enable ADC
// initUSART()
UBRR0  = F_CPU / (16 * 9600UL) - 1;                             // 9600 bps
UCSR0B = UCSR0B & ~(1<<UCSZ02);                                 //
UCSR0C = UCSR0C | (1<<UCSZ01) | (1<<UCSZ00);                    // 8-bit
UCSR0B = UCSR0B | (1<<RXEN0) | (1<<TXEN0);                      // enable RX and TX
// receiveChar()
while (!(UCSR0A & (1<<RXC0)));                                  // poll receive complete
return UDR0;
// transmitChar(char ch)
while (!(UCSR0A & (1<<UDRE0)));                                 // poll data register empty
UDR0 = ch;
// printString(char *ch)
while (*ch != '\0')                                             // until the terminator
{
    transmitChar(*ch);                                          // send this character
    ch++;                                                       // next character
}
// main loop, lines 28-29
ADCSRA = ADCSRA | (1<<ADSC);                                    // start conversion
while (ADCSRA & (1<<ADSC));                                     // wait until done
```

Q4: STDIO STREAMS (SKETCH)

`#include <stdio.h>`
`int uart_putchar(char c, FILE *s) { transmitChar(c); return 0; }`
`int uart_getchar(FILE *s) { return receiveChar(); }`
`FILE uart = FDEV_SETUP_STREAM(uart_putchar, uart_getchar, _FDEV_SETUP_RW);`
in `main`: `stdout = stdin = &uart;`
then `printf("%d\n", temp);` and `ch = getchar();` (call `getchar` only when RXC0 = 1, or it blocks the 1-second sending).

Q5: PROS AND CONS OF STDIO.H ON A MICROCONTROLLER

Pros: familiar, standard and portable code; easy formatting (`printf("%d")`); less code to write.
Cons: `printf` / `scanf` are **large** (kilobytes of the 32 KB flash), slower and use RAM; `%f` needs an extra library; the bare-register version is much smaller (the slide's Arduino-IDE C version is only **1146 bytes**).

Q6: THE C# SIDE (OUTLINE)

Form `TempForm` with a `TextBox txtTemp`, buttons `btnStart` / `btnStop`, and a **SerialPort** component (`PortName` = the Uno's COM port, `BaudRate` = 9600). `Load`: `serialPort1.Open();` · Start: `serialPort1.Write("Y");` · Stop: `serialPort1.Write("N"); txtTemp.Clear();` · `FormClosing`: `Write("N"); Close();`
`DataReceived` runs on a **different thread** from the GUI, so update controls through `Invoke`:
`string s = serialPort1.ReadLine().Trim();`
`this.Invoke((MethodInvoker)delegate { txtTemp.Text = s; });`

L. What the lecturer explained in the lectures (2020 recordings)

From the six serial-communication recordings (folders 25–30 in `Embedded Systems Lectures`). **[EXAM]** marks things he tied to the exam.

L1. THE "?" LINES ON THE SLIDES ARE EXAM QUESTIONS [EXAM]

"These statements with question marks on my slides are very important… for the exam point of view." He does **not** ask "what is baud rate?" or "what is UBRR?" (anybody can answer those). He asks things like **why all baud rates are multiples of 80**, or "using an example, explain the purpose of the **UL suffix**". The slide questions in this lesson: why not parallel; why a separate peripheral; why baud rate matters; how the receiver recognises characters; why not connect directly (line driver); UDRE or TXC for throughput; the purpose of TXC; the role of pin control; the need for shift registers; why enable/disable RX/TX; why not other baud values; why UL; what happens with a high error; how to minimise it; what a terminal emulator is; what CR and LF are.

L2. Serial vs parallel, sync vs async

- A student said "the microcontroller processes one bit at a time". No: the ATmega328P is an **8-bit** CPU and processes 8 bits at a time. The real reason for serial is **pins**: 8-bit parallel costs 16 I/O pins of the 23; an 8-pin ATtiny has only about 3 spare pins.
- Students guessed "cost of wires"; he said the cost is small compared with the speed gained, so that's not it.
- Crosstalk: the magnetic field of one wire induces current in the next, so a 0 can arrive as a 1. It is worse over long distances. Twisting the send and return wires so their fields cancel gives twisted-pair (UTP) cable.
- Synchronous explained with two clocks joined by a wire: one bit per clock pulse, and the receiver need not know the speed. Asynchronous saves that wire, but the receiver must know the rate.
- The serial standard is **RS-232** ("Recommended Standard 232").

L3. Why a separate USART: "living on another planet" [EXAM]

- Same reason as timers: the CPU is single-core, MHz not GHz, low power. Offload timing, counting, ADC, PWM and communication to peripherals and keep the CPU for computation.
- "Embedded systems programming is like living on another planet": a small oxygen tank and one bottle of water, so you save every molecule. On PCs you lived in the luxury of gigabytes and gigahertz; here you must save every bit of resource. "One particular thing I am going to assess" in the exam is whether you think this way.

L4. Frames and baud rate

- Worked at **1 bps** sending 'A' then 'B': start bit in second 0–1, LSB in 1–2… MSB in 8–9, stop in 9–10; the next start at the 11th second, stop at the 20th.
- Baud rate is named after the French engineer Émile Baudot. 9600 is the most common rate and the Arduino default.
- Parity bit: "I am not going to explain parity, you need to read and learn."
- **Why standard rates are multiples of 80 [EXAM]:** dumb terminals showed **80 characters per line**, 24 lines. 9600 bps = 960 characters/s = **12 lines per second**, half a screen. "No relationship to the word size of the microprocessor."
- A wrong baud rate makes the receiver "go mad" and the Serial Monitor prints weird characters.

L5. RS-232 and USB bridges

- **DB9** = D-shaped subminiature, 9 pins; only RXD and TXD are used for data.
- RS-232 dates from the **1960s mainframe** era (vacuum tubes, high voltages; introduced by IBM): 1 = −3 to −25 V, 0 = +3 to +25 V. Today we use TTL (2–5 V high, 0–0.8 V low) and newer CMOS uses about 3.3 V. Asked why the voltages are negative, he said honestly that you would need IBM's old documentation.
- **MAX232** (Maxim Integrated) on the **Arduino Severino** ("serial" in Italian), one of the first Arduino boards (mid-2000s).
- **FT232** (Future Technology Devices International) on the **Duemilanove** ("2009"): powered from USB, TX/RX LEDs, but **not plug-and-play** (needs a driver from ftdichip.com). That's why he told students to buy the **Uno R3**, not cheaper pre-R3 Unos.
- Uno R3: the **ATmega16U2** does USB↔UART; it has a 16 MHz **crystal** because USB needs accuracy, while the main chip uses a smaller, cheaper, less accurate **ceramic resonator**. "Uno" means one in Italian.

L6. Transmitter/receiver details [EXAM]

- "Many people know how to use the transmitter but they don't know how it works. As university students you should know."
- UDRE vs TXC: he made students reason it out. **UDRE gives maximum throughput**; TXC is for **half-duplex** (walkie-talkie "over") and **power saving** (a spy device behind enemy lines with a tiny battery: make sure the shift register has emptied

before switching the USART off). The same applies to the two interrupts.

- **Pin control** switches PD0/PD1 between GPIO and RXD/TXD. **Shift registers** are needed for parallel-in/serial-out (TX) and serial-in/parallel-out (RX).
- Transmit and receive buffers **share one address**, hence one name, UDR. "If you write, the transmit buffer is referred; if you read, the receive buffer." There is no conflict, even when sending and receiving at the same time.
- UDRE's **default is 1** (empty at start); the other flags start at 0. The baud formula's derivation (why /16) is in the datasheet (p. 227); he did not cover it.
- Reasons to enable/disable RX/TX: **save power**, and **override** GPIO only when needed.

L7. Baud error, UL and crystals [EXAM]

- 103.17 → 103 (registers hold integers) → 9615.38 bps → 0.16% (≈ 0.2%). The generally accepted limit is **2.5%**; at 10% it never works.
- UL explained bit by bit: 16 × 9600 = 153 600 (20 bits); as a 16-bit int only **22 528** remains → UBRR = **709** instead of 103. "These are logical errors, not compile errors. Very difficult to find unless you know the subject well."
- Critical systems (military, medical, automotive, avionics) need **0%**: a guided missile link error could cause friendly fire; a drone with carbon-fibre propellers crashing into a crowd. Use a precision crystal such as **18.432 MHz** → UBRR exactly 119.
- Student: changing the crystal also changes the timers? Yes: redo the timer, PWM and ADC calculations. 18 432 000 / 1024 = 18 000 exactly, so prescalers can still give whole numbers.
- Many developers "just want the functionality" and ignore worst cases. You must consider them.

L8. Code walk-through, terminals, interrupts

- Setting the character size to 8 is not needed (it is the default) but makes the code more understandable.
- He explained `while(!(UCSR0A & 0b10000000));` bit by bit, twice, for students who asked.
- Quiz: comment out the UDRE wait → the PC shows `[a` (the ']' is ignored). Comment out the RXC wait → "try it".
- Terminal emulator = software imitating a 1960s dumb terminal (TTY, teletype); 24 × 80 characters; used for debugging, sensor calibration, module configuration, data logging. The Atmel Studio terminal can save to a file; the Arduino Serial Monitor works too (separately).
- CR/LF come from typewriters: **carriage return** moves the roller back to the start of the line, **line feed** moves to the next line ("new line", `\n`).
- `Serial.begin(9600)` internally sets **UBRR = 103**, 8-bit and RX/TX on: "everything wrapped into one user-friendly function". Arduino Serial also uses **polling**; System 17 shows the interrupt way.
- Exercise 7 hint: use C#'s **SerialPort** component from the toolbox. He suggested combining it with the Interactive Application module into a project (database, Crystal Reports). He guides students but does not give answers.

12. Common mistakes

Mistake	What happens / fix
Something wired to D0/D1	Upload or communication fails. Keep D0/D1 free.
Baud rates differ (code vs monitor)	Garbage characters. Match both (9600).
`16 * 9600` without UL	UBRR0 = 709, wrong speed. Use `9600UL`.
Forgetting RXEN0/TXEN0	Nothing is sent or received (off by default).
Writing UDR0 without waiting for UDRE0	Characters lost (ignored).
Reading UDR0 without waiting for RXC0	The same old character is read again.
TX connected to TX	No data. Cross them: TX → RX.
String array without room for '\0'	Memory overwritten. Size = longest text + 1.
Wrong vector name	`USART_RX_vect`, `USART_UDRE_vect`, `USART_TX_vect`.
Updating a C# control in DataReceived directly	Cross-thread exception. Use `Invoke`.

Cheat sheet

Concepts	
Serial	2 pins (TX, RX); parallel wastes pins + crosstalk
Async	no clock; agree on baud; start 0, stop 1, LSB first
Frame	1 + 8 + 1 = 10 bits; 9600 bps = 960 chars/s
RS-232	1 = −3..−25 V, 0 = +3..+25 V → MAX232
USB	FT232 (driver) or ATmega16U2 (Uno R3)
Throughput	UDRE; TXC = all sent (half-duplex, power off)

Registers and formulas	
UBRR0	F_CPU / (16 × baud) − 1; 9600 → 103 (0.16%)
Error	(actual − desired) / desired; < 2.5%
UCSR0A	RXC0 b7, TXC0 b6, UDRE0 b5
UCSR0B	RXCIE0 b7, TXCIE0 b6, UDRIE0 b5, RXEN0 b4, TXEN0 b3, UCSZ02 b2
UCSR0C	UCSZ01 b2, UCSZ00 b1 (011 = 8-bit)
Pins / ISRs	RXD D0, TXD D1; USART_RX_vect, USART_UDRE_vect